



UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL GENERAL PACHECO

TÉCNICO SUPERIOR EN PROGRAMACIÓN

PROGRAMACIÓN III

APUNTE TEORICO

<<< >>>

ADO.NET I

Prof. ABEL OSCAR FAURE

ADO.NET I

ADO.NET – CAPA CONEXIÓN

El Framework .NET define una serie de espacios de nombres, que permiten interactuar directamente con Bases de Datos Locales y Remotas.

Estos espacios de nombres se lo conocen como ADO.NET, el Framework .NET es compatible con varios proveedores de datos, cada uno esta optimizado para comunicarse con un DBMS (Gestor de Base de Datos) especifico, por Ej. (Microsoft SQL Server, Oracle, MySQL).

ADO.NET

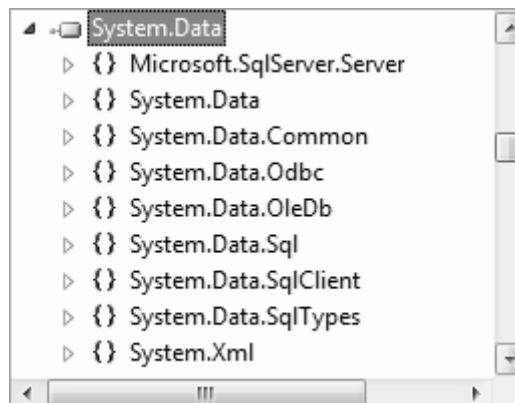
Si trabajo anteriormente con una plataforma Microsoft basada en **Component Object Model (COM)** los modelos de acceso a datos (Objetos de Active Data, o ADO) tienen poco que ver uno con el otro más allá de la A, D Y O.

Aunque ambos comparten el concepto de Objetos de Conexión y Objetos de Comandos, ya no existe el conjunto de registro.

ADO.NET fue pensado y construido básicamente con el mundo desconectado en mente, utilizando un DataSets (Conjunto de Datos). Este DataSets representa una copia local de N tablas de datos relacionadas y cada una de ellas posee una colección de filas y columnas. Utilizando el DataSets, la llamada del assembly (Ej: pagina Web o ejecutable de escritorio) puede manipular y actualizar el DataSets sin estar conectado a la Base de Datos y luego enviar estos datos modificados para que sean procesados utilizando un Data Adapter (Adaptador de Datos).

La principal diferencia entre el ADO Y ADO.NET es que este último es una biblioteca administrada de código, y se rige por las mismas reglas que cualquier otra biblioteca de códigos. Los tipos que componen ADO.NET utilizan el protocolo de gestión de memoria del CLR, lo mismo que clases, interfaces, enumeraciones, estructuras, etc.

Gran parte del espacio de nombres que representan a un tipo de proveedor determinado en ADO.NET, se encuentra en un archivo binario de nombre System.Data.dll.



LAS DOS CARAS DE ADO.NET

Se puede utilizar las bibliotecas de ADO.NET de tres maneras conceptualmente únicas: Conectado, Desconectado y a través de la Entity Framework.

MODO CONECTADO

En este modo el código base suele conectarse y desconectarse de la Base de Datos. Cuando se utiliza ADO.NET en este modo suele comunicarse con la Base de Datos a través de los Objetos Connection, Objetos Command y Objetos Data Reader.

MODO DESCONECTADO

En este modo le permite manejar un conjunto de objetos DataTable (Contenidos dentro de un DataSet) que funciona como una copia de la Base de Datos del lado del Cliente.

Cuando se obtiene un DataSet a través de un Objeto Data Adapter relacionado, la conexión se abre y cierra automáticamente. Una vez que se recibe el DataSet este puede ser manipulado sin costo de tráfico de red. Cuando el usuario decide guardar los cambios, son los Objetos Data Adapter (En conjunción con un conjunto de Instrucciones SQL) que llevan acabo la actualización de la Base de Datos, en este punto se vuelve a abrir la conexión y una vez actualizada la Base de Datos se cierra automáticamente.

PROVEEDORES DE DATOS EN .NET (DATA PROVIDER)

Un proveedor de datos .NET es un conjunto de clases que permiten acceder a los datos que son gestionados por un servidor de base de datos en particular. Todos los proveedores de datos .NET deben incluir las clases básicas para la creación de los cuatro tipos de Objetos listados en la siguiente tabla.

OBJETOS DATA PROVIDER .NET

OBJETOS	DESCRIPCIÓN
Connection	<i>Establece una conexión con una Base de Datos.</i>
Command	<i>Representa una sentencia SQL que se puede ejecutar en la Base de Datos.</i>
Data reader	<i>Provee acceso de solo lectura y solo hacia delante, a los datos de una Base de Datos.</i>
Data adapter	<i>Proporciona el enlace entre los objetos Command y Connection y un objeto Dataset.</i>

En la segunda tabla se enumeran los cuatro proveedores de datos que vienen con el .NET Framework.

Los proveedores de datos de SQL Server están diseñados para proveer acceso eficiente a una base de datos Microsoft SQL Server.

OLE DB es el proveedor de datos genérico que puede acceder a cualquier base de datos que soporta la industria Standard interfaz OLE DB. Aunque se puede usar el proveedor de datos OLE DB para acceder a una base de datos de SQL Server, usted no debe hacer eso a menos que usted planea migrar los datos a otra base de datos desde el proveedor de datos de SQL Server está optimizado para el acceso a datos de SQL Server.

El proveedor de ODBC le permite acceder a cualquier base de datos que puede trabajar con ODBC, otra interfaz de la industria de bases de datos estándar.

El proveedor de Oracle le permite acceder a datos almacenados en bases de datos Oracle.

PROVEEDORES DE DATOS INCLUIDOS EN EL .NET FRAMEWORK

PROVEEDOR	NAMESPACE	DESCRIPCIÓN
SQL Server	<i>System.Data.SqlClient</i>	<i>Permite acceder a datos Sql Server.</i>
OLE DB	<i>System.Data.OleDb</i>	<i>Permite acceder a cualquier Base de Datos que soporta OLE DB.</i>
ODBC	<i>System.Data.Odbc</i>	<i>Permite acceder a cualquier Base de Datos que soporta ODBC.</i>
Oracle	<i>System.Data.OracleClient</i>	<i>Permite acceder a Base de Datos Oracle.</i>

En la tercera tabla se muestran los nombres de las clases que se utilizan para crear objetos utilizando el servidor SQL Server, OLE DB, ODBC o proveedores de Oracle. Estas clases utilizan prefijos (SQL, OLEDB, ODBC, Oracle) para indicar el proveedor que se utiliza en cada clase.

NOMBRES DE CLASES PARA CADA PROVEEDOR DE DATOS

OBJETOS	SQL Server	OLE DB	ODBC	Oracle
Connection	<i>SqlConnection</i>	<i>OleDbConnection</i>	<i>OdbcConnection</i>	<i>OracleConnection</i>
Command	<i>SqlCommand</i>	<i>OleDbCommand</i>	<i>OdbcCommand</i>	<i>OracleCommand</i>
Data reader	<i>SqlDataReader</i>	<i>OleDbDataReader</i>	<i>OdbcDataReader</i>	<i>OracleDataReader</i>
Data adapter	<i>SqlDataAdapter</i>	<i>OleDbDataAdapter</i>	<i>OdbcDataAdapter</i>	<i>OracleDataAdapter</i>

COMO TRABAJAN LOS COMPONENTES BASICOS EN ADO.NET

En la figura se muestran los principales componentes de ADO.NET que se utilizan para trabajar con datos en una aplicación de Windows.

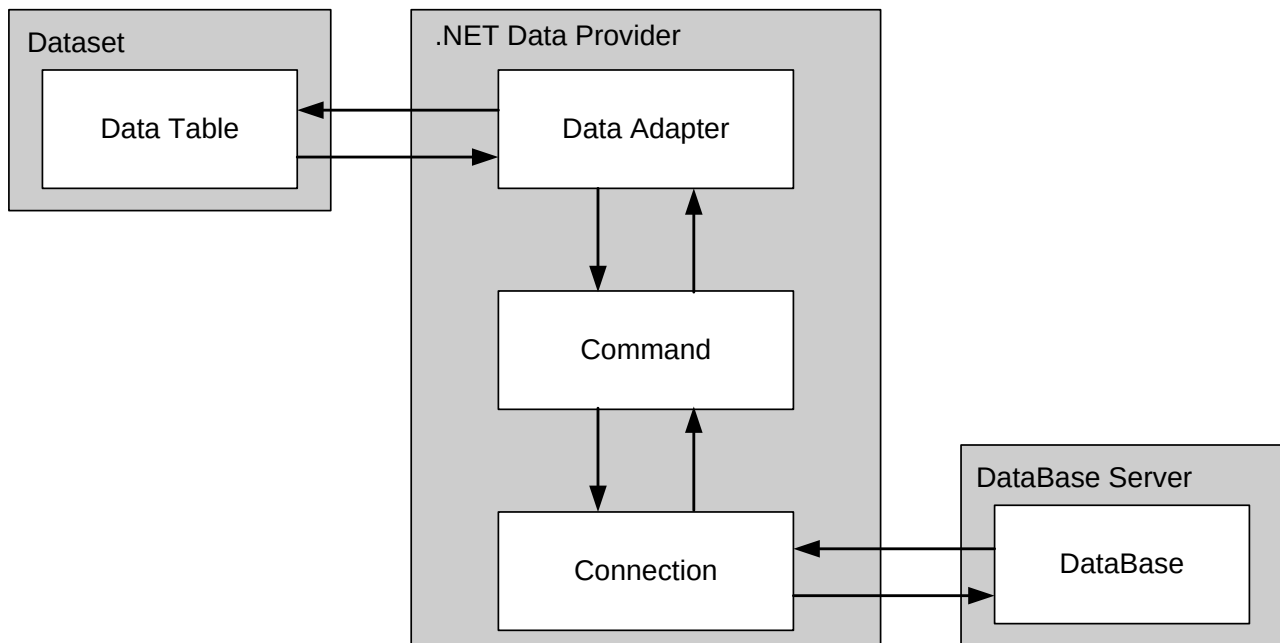
Para empezar, los datos utilizados por una aplicación se almacenan en un Dataset que contiene una o más tablas de datos. Para recuperar los datos de la base de datos y cargarlos en una tabla de datos, se utiliza un DataAdapter. La función principal del DataAdapter es la gestión del flujo de datos entre un Dataset y una base de datos. Para ello se utilizan los comandos (Command) que definen las instrucciones SQL para ser emitidos. El comando (Command) para la recuperación de datos, por ejemplo, por lo general define una instrucción de selección (Select). A continuación, el comando (Command) se conecta a la base de datos usando una conexión (Connection) y pasa a la instrucción de selección a la base de datos. Después de que la instrucción SELECT se ejecuta, el conjunto de resultados que produce se envía de nuevo al DataAdapter, que almacena los resultados en las tablas de datos.

Para actualizar los datos en una base de datos, el DataAdapter determina que fila de las tablas de datos serán insertadas, actualizadas o borradas. A continuación, utiliza los comandos que definen la inserción, actualización y delete para la tabla de datos para actualizar las filas asociadas en la base de datos.

Al igual que el comando que recibe datos de la base de datos, los comandos de la base de datos update utilizan una conexión para conectarse a la base de datos y llevar a cabo la operación solicitada.

Aunque esto no es evidente en esta figura, los datos en un Dataset es independiente de la base de, la conexión a la base de datos está típicamente cerrado después de que los datos se recuperan de la base de datos. Entonces, la conexión se vuelve a abrir cuando sea necesario. La aplicación debe trabajar con la copia de los datos que se almacenan en la base de datos. La arquitectura que se utiliza para implementar este tipo de procesamiento de datos se conoce como la arquitectura de datos desconectados.

Aunque este enfoque es más complicado que una arquitectura conectada, tiene varias ventajas. Una ventaja es que el uso de la arquitectura desconectada puede mejorar el rendimiento del sistema debido a la utilización de los pocos recursos del sistema para mantener las conexiones. Otra ventaja de ADO.NET es que sea compatible con aplicaciones Web ASP.NET, que son inherentemente desconectadas.



CONCURRENCIA Y LA ARQUITECTURA DE DATOS DESCONECTADOS

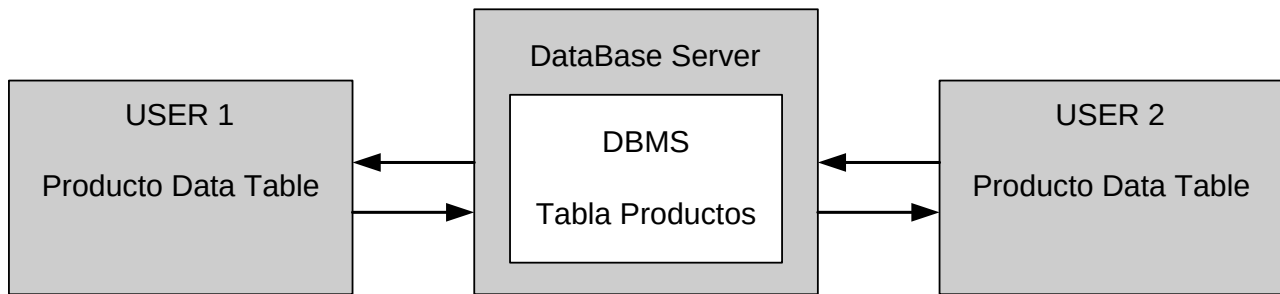
Aunque la arquitectura de datos desconectado tiene ventajas, también tiene algunas desventajas. Uno de ellos es el conflicto que puede ocurrir cuando dos o más usuarios recuperan y luego tratan de actualizar los datos en la misma fila de una tabla. Esto se llama un problema de simultaneidad, es posible porque una vez que un programa recupera los datos de una base de datos, la conexión a la base de datos se deja caer, como resultado, el sistema de gestión de base de datos no puede gestionar el proceso de actualización. Para ilustrar, Considerar la situación mostrada en la figura. Aquí, dos usuarios han recuperado la tabla de productos a partir de una base de datos, por lo que una copia de la tabla de productos se almacena en la PC de cada usuario. Estos usuarios pueden utilizar el mismo programa o dos programas diferentes. Ahora, supongamos que el usuario 1 modifica el precio unitario en la fila para ADV3 de productos y actualizaciones de la tabla productos en la base de datos. Y supongamos que el usuario 2 modifica la descripción de la fila para el mismo producto, a continuación, intenta actualizar la tabla de productos en la Base de Datos. ¿Qué pasará?

Cuando se utiliza ADO.NET, usted tiene dos opciones para el control de concurrencia. De manera predeterminada, un programa utiliza simultaneidad optimista, que comprueba si una fila se ha modificado desde que se recuperó en caso afirmativo, la actualización o supresión será rechazado y una excepción de concurrencia se tirará. Entonces, el programa debe controlar el error. Por ejemplo, podría mostrar un mensaje de error que indica que el usuario NO puede hacer nuevo Cambio.

Por el contrario, la técnica de trabajo el "último gana", funciona de la manera que su nombre indica. Ya que la comprobación no se realiza con esta técnica, la fila que actualiza el último usuario sobrescribe los cambios realizados en la fila por un usuario anterior. Para el ejemplo anterior, la fila actualizada por el usuario 2 se superponen a los cambios realizados por el usuario 1, lo que significa que la descripción sea correcta, pero el precio por unidad será incorrecto. Ya que los errores como estos, provocan datos corruptos en una base de datos, la simultaneidad optimista es utilizada por la mayoría de los programas, lo que significa que sus programas tienen que manejar las excepciones de concurrencia que se generan.

Si usted sabe que la concurrencia será un problema, puede utilizar un par de técnicas de programación para limitar las excepciones de concurrencia. Si un programa utiliza un conjunto de datos, una técnica es actualizar la base de datos con frecuencia para que

otros usuarios puedan recuperar los datos actuales. El programa también debería actualizar su base de datos con frecuencia para que contenga los cambios recientes realizados por otros usuarios. Otra manera de evitar las excepciones de concurrencia es recuperar y trabajar con una sola fila a la vez. De ese modo, es menos probable que dos usuarios actualizarán la misma fila al mismo tiempo. Por el contrario, si dos usuarios recuperan la misma tabla, por supuesto, recuperar las mismas filas. Entonces, si ambos actualizan la misma fila de la tabla, aunque puede que no sea al mismo tiempo, una excepción de concurrencia se producirá cuando se intenta actualizar la base de datos.

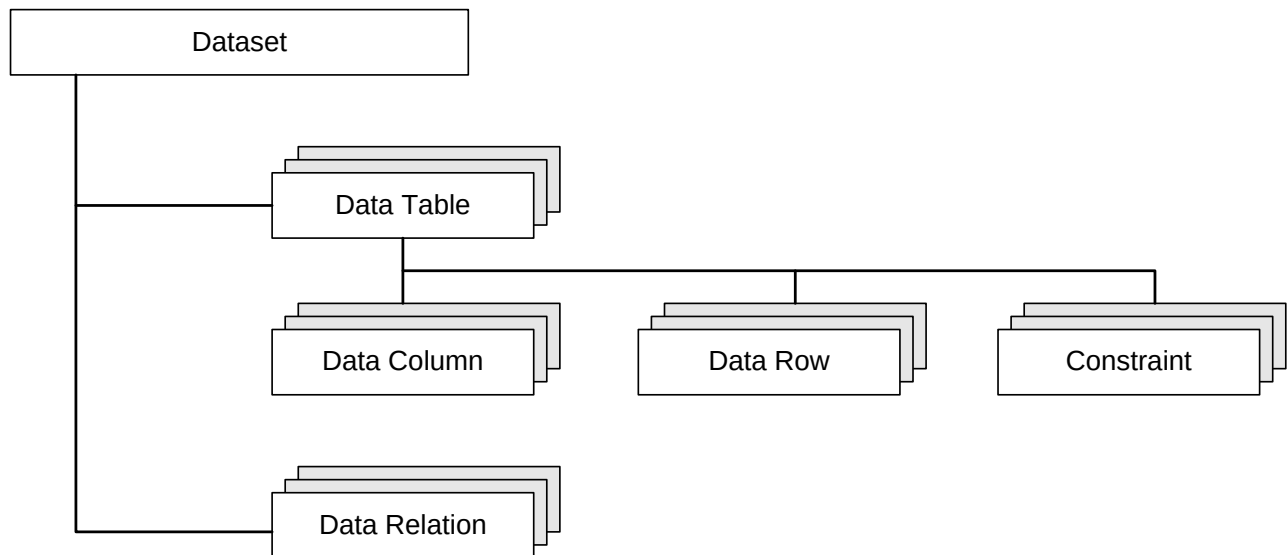


COMO ESTA ORGANIZADO UN DATASET

Ahora que usted tiene una idea general de cómo las clases del proveedor de datos proporcionan acceso a una base. Usted necesita saber más de la parte ADO.NET arquitectura desconectada: Dataset. En la figura se ilustra la organización básica de un Dataset ADO.NET. Lo primero que debe observar en esta figura es que un Dataset de base de datos se estructura muy similar a una Base de Datos relacional. Puede contener una o más tablas, y cada tabla puede contener una o más columnas y filas.

Además, cada tabla posee una clave externa de otra tabla en la base de datos. Si un Dataset contiene dos o más tablas, el Dataset también puede definir las relaciones entre las tablas. Al pensar en un Dataset su estructura es muy similar a una base de datos relacional, es importante darse cuenta de que cada tabla en un Dataset se corresponde con el conjunto de resultados que regresa una instrucción de selección no necesariamente a una tabla real en una base de datos. Por ejemplo, una instrucción SELECT puede unir datos de varias tablas en una base de datos para producir un solo conjunto de resultados. En este caso, la tabla del conjunto de datos que representan datos de cada una de las tablas que participan en la unión.

Usted también debe saber que cada grupo o un objeto en el diagrama de esta figura es almacenado en una colección, todas las columnas de una tabla, por ejemplo, se almacenan en una colección de columnas, y todas las filas se almacenan en una colección de filas.



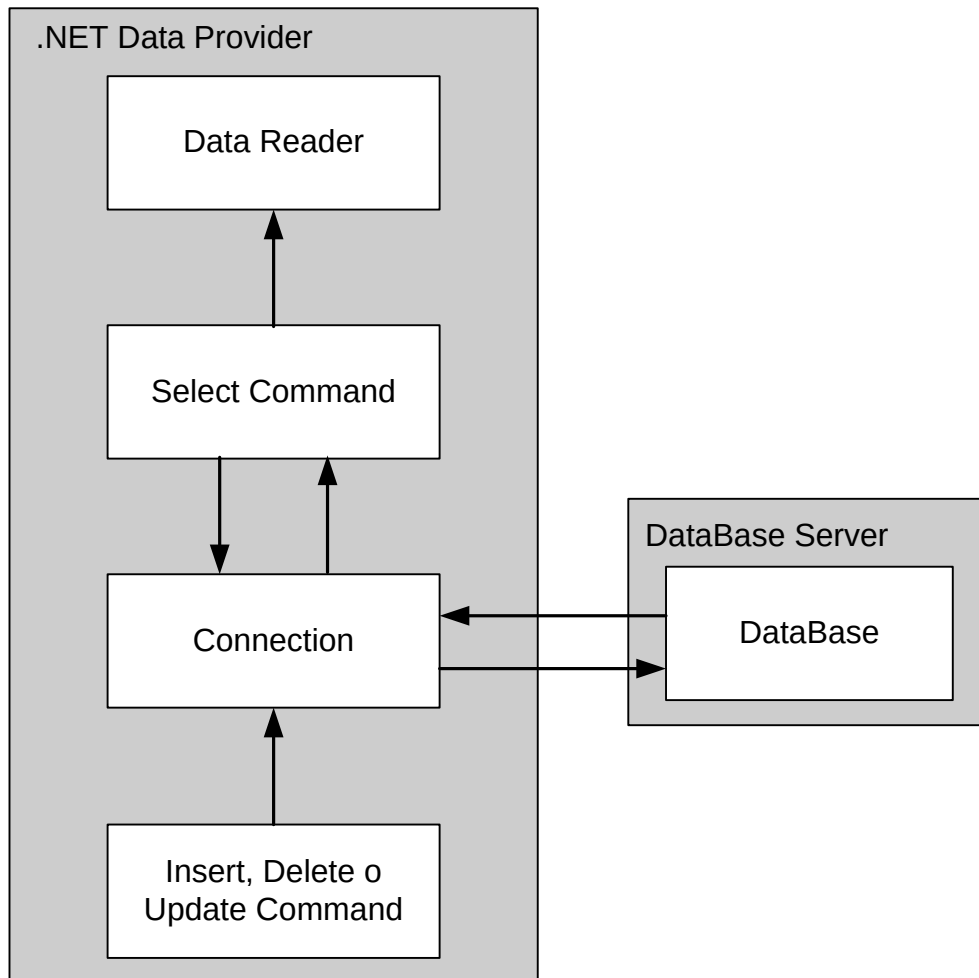
COMO TRABAJAR CON DATOS SIN UTILIZAR UN DATA ADAPTER

Cuando se quiere trabajar con dos o más filas de una base de datos, al mismo tiempo, normalmente se utiliza un adaptador de datos (Data Adapter) para recuperar las filas y almacenarlos en un Dataset.

Sin embargo, también puede trabajar con los datos en una base de datos sin necesidad de utilizar un adaptador de datos (Data Adapter). La figura muestra cómo hacerlo.

Como se puede ver, se utilizan objetos de comando (Command) y la conexión (Connection) para acceder a la base de datos. En lugar de utilizar un adaptador de datos (Data Adapter) para ejecutar los comandos, se ejecutan las órdenes directamente. Cuando se hace esto, usted también tiene que proporcionar el código para manejar el resultado del comando. Si emite un comando que contiene una sentencia de inserción, actualización o eliminación, por ejemplo, el resultado sería un entero que indica el número de filas que se vieron afectados por la operación. Usted puede utilizar esa información para determinar si la operación fue un éxito.

Si se ejecuta un comando que contiene una instrucción SELECT, el resultado es un conjunto que contiene las filas requeridas. Para leer a través de las filas del conjunto de resultados, se utiliza un objeto Data Reader. Aunque un Data Reader proporciona una forma eficiente de la lectura de las filas de un conjunto de resultados, no se puede utilizar para modificar las filas. Además, sólo se le permite leer las filas en una dirección. Una vez que lea la siguiente fila, la fila anterior no está disponible. Por eso, normalmente se utiliza un Data Reader para recuperar y trabajar con una única fila de la base de datos a la vez.



DOS FORMAS DE CREAR OBJETOS DE ADO.NET

En las figuras se muestran dos técnicas básicas que puede utilizar para crear los objetos de ADO.NET que necesita para desarrollar aplicaciones de base de datos.

Primero, puede crear objetos de ADO.NET a partir de un origen de datos, este aparece en la ventana Orígenes de datos. Las fuentes de datos están disponibles desde .NET 2.0 y hacen que sea fácil crear rápidamente formularios que trabajan con los datos de un origen de datos como una base de datos.

En este ejemplo, la fuente de datos corresponde con los datos de la tabla Productos en la base de datos Neptuno. Por ahora, usted debe saber que una vez que se crea una fuente de datos, puede arrastrarlo a un formulario para mostrar automáticamente los controles sobre el formulario y para crear los objetos de ADO.NET para trabajar con los datos del origen de datos. En esta figura, por ejemplo, puede ver los controles y los objetos que se generan cuando se arrastra la tabla de Productos en el formulario. En este caso, un control DataGridView ha sido añadido al formulario para exhibir los productos de la tabla de Productos, y una barra de herramientas se ha añadido que le permite trabajar con estos datos.

Además, cinco objetos han sido añadidos a la bandeja diseñador de componentes por debajo del Formulario. Tres de ellas son objetos de ADO.NET. El primero, llamado neptunoDataSet, define el conjunto de datos para el formulario. Entonces, un objeto denominado ProductosTableAdapter define el adaptador de la tabla para la tabla Productos. Un adaptador de la tabla es similar a un adaptador de datos, pero sólo puede ser genera-

do por el diseñador. Por último, un objeto denominado TableAdapterManager se asegura de que si dos o más tablas relacionadas son actualizadas por el formulario, la integridad referencial se mantiene.

Los otros dos objetos en la bandeja de diseño de componentes se utilizan para enlazar los controles en el formulario con la fuente de datos. El primer objeto, con el nombre ProductosBindingSource identifica la tabla de productos de la fuente de datos con los controles.

El segundo objeto, llamado productosBindingNavigator, define la barra de herramientas que aparece en la parte superior del formulario. A pesar de que por lo general no necesita cambiar las propiedades de los objetos en la bandeja del diseñador de componentes, usted debe saber que usted puede hacer eso con la misma técnica que se utiliza para cambiar las propiedades de un control en un formulario. Es decir, sólo tiene que seleccionar un objeto para mostrar sus propiedades en la ventana de propiedades y luego trabajar con ellos desde allí.

La segunda técnica para la creación de objetos ADO.NET es escribir el código manualmente. El código que se muestra en esta figura, por ejemplo, se crean cuatro objetos: una conexión, un comando llamado SelectCommand que contiene una instrucción Select, un adaptador de datos denominado ProductosDataAdapter, y un Dataset denominado ProductosDataSet.

Aunque la creación de objetos de ADO.NET a través del código es más lento que el uso de fuentes de datos, puede dar lugar a un código más compacto y eficiente.

